

Interfaces Gráficas

Qué es SWING

Es un framework MVC surgido en 1998 que se utiliza para desarrollar aplicaciones gráficas, siendo independiente de la plataforma. Se considera una buena herramienta para entender cómo funcionan las interfaces gráficas, además de que emplea en su estructura conceptos de POO: clases, objetos, encapsulación, herencia, polimorfismo...

Componentes básicos:

Componente	Clase
Ventana	<code>JFrame</code>
Panel	<code>JPanel</code>
Etiqueta	<code>JLabel</code>
Botón	<code>JBUTTON</code>
Campo de texto	<code>JTextField</code>
Área de texto	<code>JTextArea</code>
Lista desplegable	<code>JComboBox</code>
Tabla	<code>JTable</code>
Checkbox	<code>JCheckBox</code>

`JFrame`: la base de nuestra interfaz sobre la que vamos a construir el resto de nuestros componentes.

`JPanel`: es un contenedor invisible para organizar y agrupar componentes

`JLabel`: sirve para mostrar textos simples

`JButton`/`JCheckBox`/`JRadioButton`: componentes para interactuar y conectarlos a eventos para que realicen funciones concretas

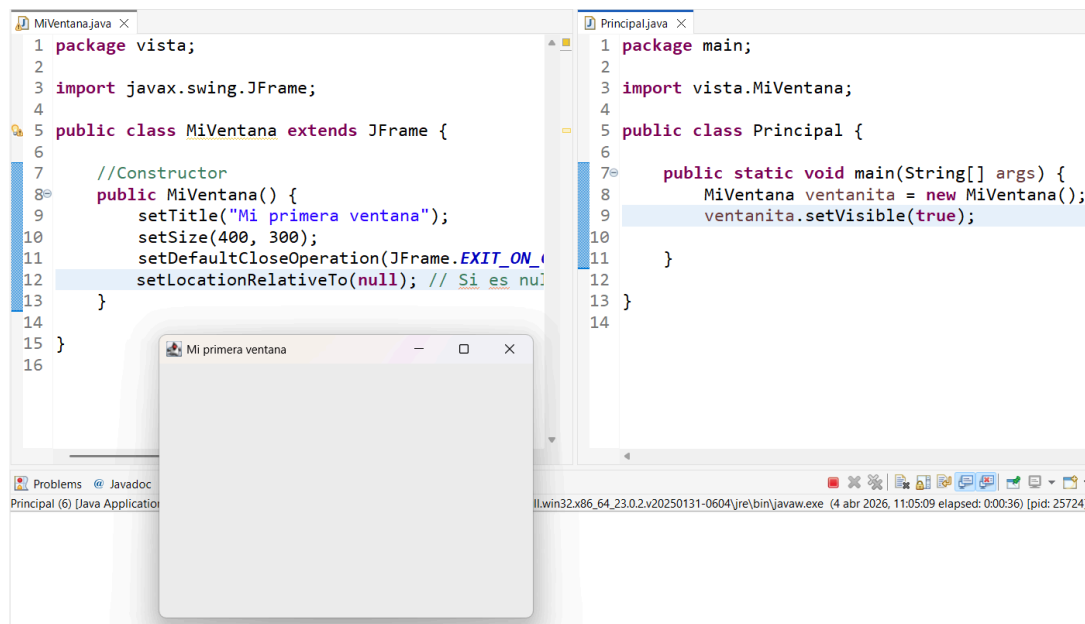
`JTextField`/`JTextArea`: componentes usados para introducir un valor de texto y capturarlo

No es la opción más moderna para la creación de un entorno gráfico. De hecho, en la actualidad normalmente se usa java (con SpringBoot) como el backend (la parte de la lógica) y se usa algún framework de JS (React, Vue, Angular) para la parte de contacto con el usuario, lo que se conoce como frontend. Sin embargo, como ya se ha mencionado anteriormente, es una buena manera de entender cómo funcionan las interfaces gráficas desde una aproximación de la POO.

<https://docs.oracle.com/en/java/javase/11/docs/api/java.desktop/javax/swing/JFrame.html>

Crear una ventana de manera manual

Dado que, como hemos dicho, las interfaces gráficas están integradas en la POO podemos crear una clase que herede de la clase base que nos permitirá crear nuestra interfaz.



Como se ve en la imagen, esta es la estructura mínima para poder crear una interfaz. Estamos creando un objeto dentro de nuestro main y le damos la característica de que sea visible. El resto es simplemente utilizar métodos de la clase AWT (la padre de JFrame) para poder dotar de características físicas a nuestra ventana.

Instalar Window Builder

Help → Eclipse MarketPlace → WindowBuilder

Eclipse Marketplace

Select solutions to install. Press Install Now to proceed with installation.
Press the "more info" link to learn more about a solution.



Search

Recent

Popular

Installed

Research at the Eclipse

Find:



WindowBuilder Current

WindowBuilder is composed of SWT Designer and Swing Designer and makes it very easy to create Java GUI applications without spending a lot of time writing code.... [more info](#)

by [Eclipse Foundation](#), EPL 2.0
[SWT](#) [swing](#) [wysiwyg](#) [graphical](#) [WindowBuilder](#)

1094



Installs: **1,33M** (7.613 last month)

Crear una ventana con window Builder

 New

Select a wizard

Create an empty JFrame

Wizards:

- > General
- > Git
- > Gradle
- > Java
- > Maven
- > Oomph
- > Plug-in Development
- > Tasks
- > User Assistance
- ▼ WindowBuilder
 -  Project Palette
 - ▼ Swing Designer
 -  (Deprecated) JApplet
 -  Application Window
 -  JDialog
 -  **JFrame**
 -  JInternalFrame
 -  JPanel
 -  Swing Automatic Databinding
 - > SWT Designer

```

1 import java.awt.EventQueue;
2
3 public class MiVentanaDos extends JFrame {
4
5     private static final long serialVersionUID = 1L;
6     private JPanel contentPane;
7
8     public static void main(String[] args) {
9         EventQueue.invokeLater(new Runnable() {
10             public void run() {
11                 try {
12                     MiVentanaDos frame = new MiVentanaDos();
13                     frame.setVisible(true);
14                 } catch (Exception e) {
15                     e.printStackTrace();
16                 }
17             }
18         });
19     }
20
21     public MiVentanaDos() {
22         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
23         setBounds(100, 100, 450, 300);
24         contentPane = new JPanel();
25         contentPane.setBorder(new EmptyBorder(5, 5, 5, 5));
26         setContentPane(contentPane);
27     }
28 }

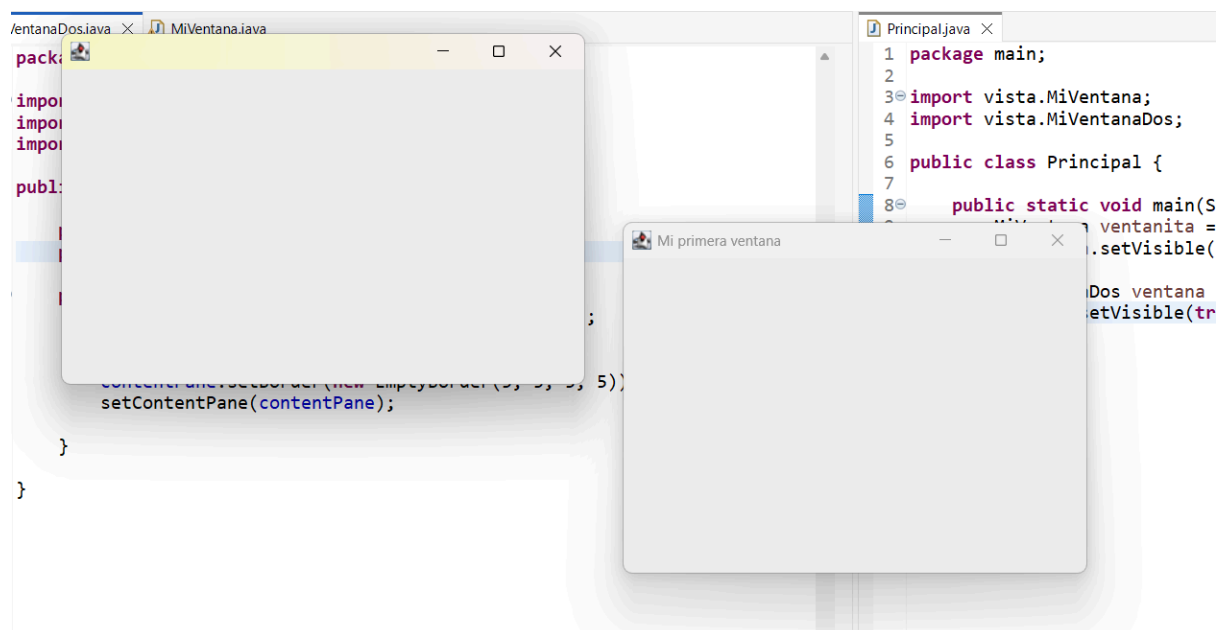
```

```

1 package main;
2
3 import vista.MiVentana;
4 import vista.MiVentanaDos;
5
6 public class Principal {
7
8     public static void main(String[] args) {
9         MiVentana ventanita = new MiVentana();
10        ventanita.setVisible(true);
11
12        MiVentanaDos ventana = new MiVentanaDos();
13        ventana.setVisible(true); //Eliminamos primero el otro main
14    }
15
16 }
17
18

```

Si lo creamos de manera automática el propio eclipse nos crea un método main para poder invocar a la clase de manera interna. Podemos eliminar este main y quedarnos con el que tenemos nosotros. Si nos fijamos, en este caso nos ha creado un panel automático para poder colocar componentes.



Aparte de poder crear algunas características de manera más rápida y directa, una de las principales ventajas de window builder es que nos permite usar una interfaz donde poder situar los componentes y las capas como queramos. Se despliega si le damos a la pestaña de design, en la parte inferior:

```
MiVentanaDos.java x MiVentana.java
1 package vista;
2
3 import javax.swing.JFrame;
4 import javax.swing.JPanel;
5 import javax.swing.border.EmptyBo
6
7 public class MiVentanaDos extends
8
9     private static final long ser
10 private JPanel contentPane;|
11
12 public MiVentanaDos() {
13     setDefaultCloseOperation(
14     setBounds(100, 100, 450,
15     contentPane = new JPanel(
16     contentPane.setBorder(new
17     setContentPane(contentPan
18
19 }
20
21 }
22
```

Source Design

Structure

Components

- (javax.swing.JFrame)
 - contentPane

Properties

Variable	contentPane
Layout	(java.awt.FlowLayout)
Class	javax.swing.JPanel
background	240,240,240
border	EmptyBorder
foreground	0,0,0
tab order	
toolTipText	

Palette

System

- Selection
- Marquee
- Choose component
- Tab Order
- Containers
 - JPanel
 - JScrollPane
 - JSplitPane
 - JTabbedPane
 - JToolBar
 - JLayeredPane
 - JDesktopPane
 - JInternalFrame
- Layouts
 - Absolute layout
 - FlowLayout
 - BorderLayout
 - GridLayout
 - GridBagLayout
 - CardLayout
 - BoxLayout
 - SpringLayout
 - FormLayout
 - MigLayout
 - GroupLayout
- Struts & Springs
- Components
 - JLabel
 - JTextField
 - JComboBox
 - JButton
 - JCheckBox
 - JRadioButton

¿Qué es un evento?

Es un suceso que ocurre como reacción de la interacción del usuario con algún componente de la interfaz gráfica, haciendo que suceda algo (cambio de color, nueva ventana, que aparezca un nuevo texto...). Puede producirse como consecuencia de pulsar un botón, deslizar una barra, activar un checkbox...

Un evento es un objeto lanzado por otro y enviado a un tercer objeto que se llama escuchador (listener). El listener está preparado, esperando que se le envíe un mensaje para realizar una acción (En POO los mensajes también son objetos). Un evento se lanza (o se dispara) cuando ocurre una determinada situación.

Para que ocurra un evento, por tanto, necesitamos tres objetos distintos: el objeto fuente - el encargado de lanzar los eventos, lanzará una serie de métodos asociados al tipo de objeto que sea-, el objeto listener -recibe el evento, lo captura y ejecuta el código correspondiente- y el objeto evento - el objeto que será enviado desde el objeto fuente hasta el objeto listener-.

Los eventos tienen asociados una interfaz para manejar el evento. Si queremos que nuestra aplicación realice cualquier tipo de evento necesitamos implementar las interfaces destinadas a capturar los eventos. Estas interfaces son conocidas como listeners, y están a la espera de que se produzca un evento.

Por ejemplo, el método *actionPerformed* es el encargado de gestionar eventos del tipo **ActionEvent** (eventos de acción, se producen, por ejemplo, al hacer clic en un botón). Este método está implementado en la interfaz ActionListener (implementa escuchadores de eventos de acción). El código de ese método será invocado automáticamente cuando el objeto fuente produzca un evento de acción.

<https://docs.oracle.com/en/java/javase/11/docs/api/java.desktop/java/awt/event/ActionListener.html>

<https://docs.oracle.com/javase/tutorial/uiswing/events/actionlistener.html>

Algunos eventos clásicos son por ejemplo de componentes (ActionEvent) como pulsar un botón o introducir un texto, usar el ratón (MouseEvent), gestionar una ventana (WindowEvent), o usar alguna de las teclas del teclado (KeyEvent). Hay muchos otros eventos que están asociados a métodos con interfaces concretas.

Por ejemplo, para capturar algo relacionado con el uso del ratón usaría el objeto fuente MouseEvent; depende de lo que queramos capturar usaríamos un método u otro (cuando se pulsa un botón mousePressed(), cuando se libera un botón mouseReleased()...). Por último estaría la interfaz asociada a este evento, la que captura el evento, que sería la MouseListener.

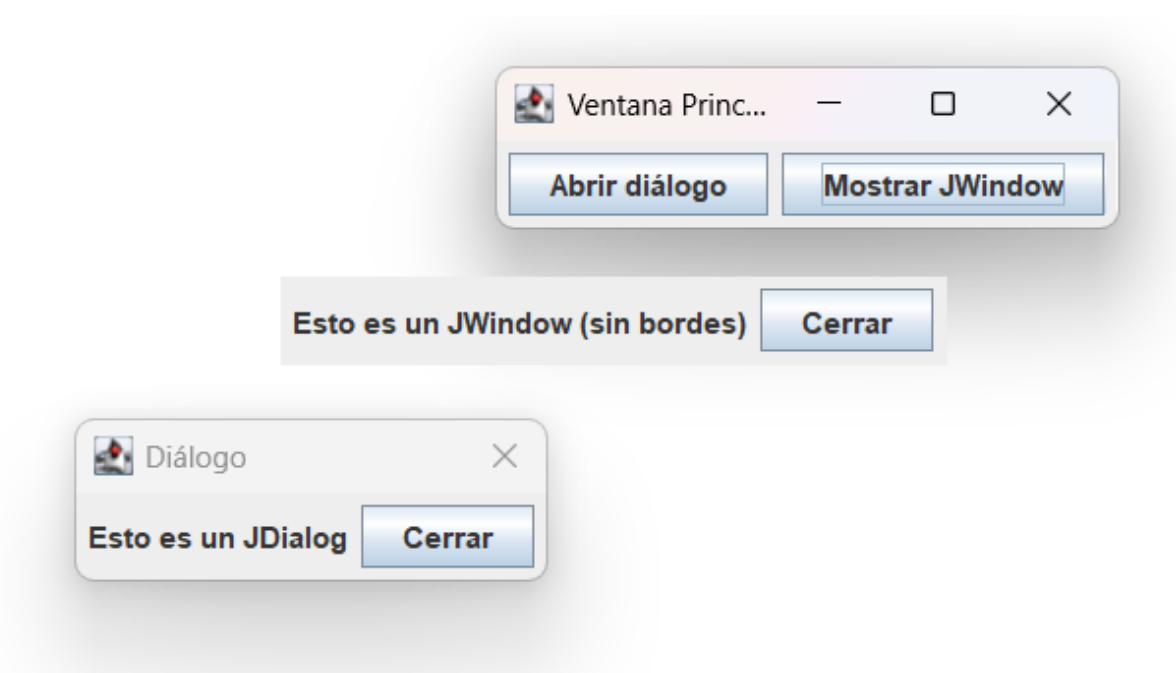
Ejemplo calculadora.

¿Qué diferencia hay entre JFrame, JDialog y JWindow?

JFrame es la ventana principal de una aplicación, será la base sobre la cual construiremos nuestra aplicación gráfica. Podemos, como hemos visto, colocarle un título, unas dimensiones (veremos en el ejemplo la diferencia entre los métodos `setSize()` y `pack()`), unos bordes...

JDialog es una ventana secundaria que depende siempre de otra ventana, normalmente de un JFrame. Es una ventana que se usa para mensajes o confirmaciones, como una ventana emergente (al estilo de `JOptionPane`). Puede ser Modal (bloquea la ventana principal hasta que se cierre) o no modal (permite seguir usando la ventana principal).

JWindow es una ventana sin decoraciones, no tiene barra de título, ni botones ni bordes. Su uso principal es para ventanas de carga o para notificaciones visuales personalizadas, es decir, como si usáramos una especie de capa flotante limpia.

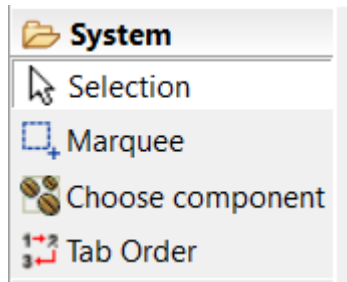


Ejemplo Swing.

Principales Componentes

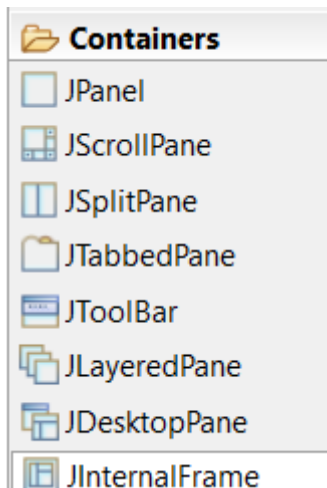
<https://academiasanroque.com/guia-basica-sobre-componentes-de-java-swing/>

<https://www.universojava.com/2022/10/componentes-y-ventanas-de-java-swing.html>



System: herramientas fundamentales de navegación y selección

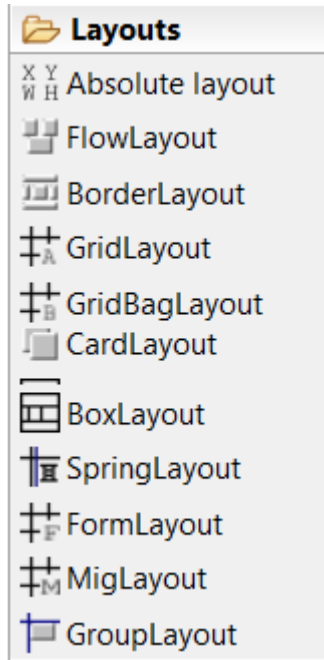
- Selection: Cursor estándar
- Marquee: Para seleccionar diferentes elementos a la vez
- Choose component: Para buscar un componente específico cuando hay muchos elementos en una interfaz; puede utilizarse también para añadir componentes no visibles en la paleta.
- Tab order: Sirve para definir el foco en los elementos de la interfaz.



Containers: son contenedores para guardar y situar otros componentes

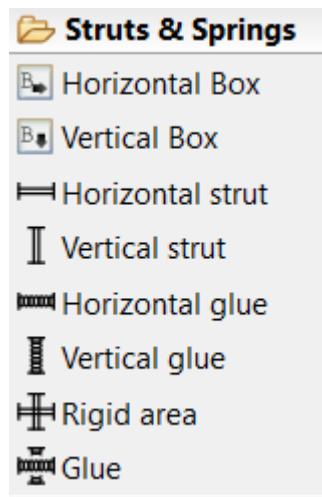
- JPanel: contenedor más básico, organiza y agrupa componentes. Se pueden usar varios para ordenar la ventana en diferentes espacios. Cuidado, si queremos poder reordenar nuestros componentes de manera libre debemos cambiar su layout a absolutelayout.
- JScrollPane: añade barras de desplazamiento a un componente cuando su contenido es más grande que el espacio visible.
- JSplitPane: divide la ventana en dos partes ajustables.

- JTabbedPane: permite organizar el contenido en pestañas, cada una muestra un panel diferente.
- JToolBar: es una barra de herramientas, para colocar botones de acceso rápido.
- JLayeredPane: permite colocar componentes en capas, controlando cuál se ve delante o detrás. Permite dar control sobre el orden de apilamiento de los componentes.
- JDesktopPane: permite tener varias ventanas internas dentro de una principal.
- JInternalFrame: las ventanas internas dentro del panel anterior.

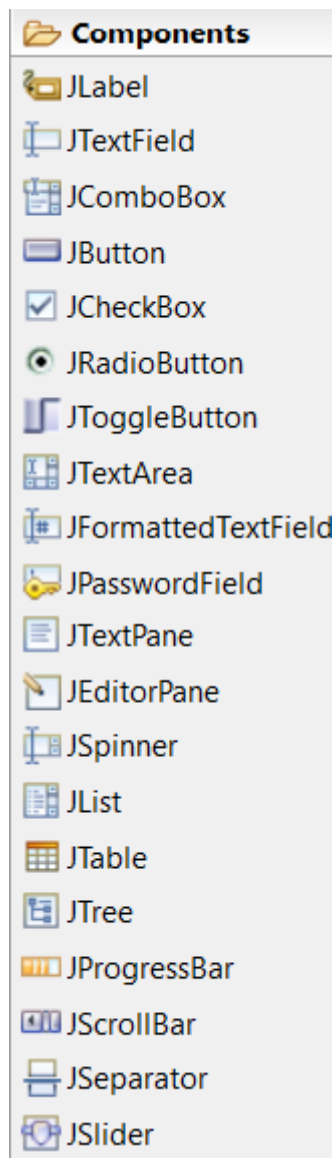


Layouts: son reglas de organización, permiten reajustar el comportamiento del diseño de los componentes en función de las propiedades que tengan (por ejemplo: si se cambia el tamaño de la ventana decide si los botones se estiran o se quedan quietos).

- Absolute: los componentes se colocan de manera fija, pero no cambian si se reajusta la ventana.
- Flow: los componentes se colocan uno al lado de otro, como palabras en un renglón.
- Border: divide el contenedor en 5 zonas: norte, sur, este, oeste y centro.
- Gridbag: cuadrícula avanzada, potente y complejo
- Card: Solo se ve un componente a la vez y se puede cambiar de uno a otro.
- Box: alinea los componentes en una sola fila o en una columna.
- Spring: Define la posición mediante posiciones relativas a otros componentes..
- Form: especializado en formularios, alinea las etiquetas con sus cuadros de texto.
- Mig: Flexible y fácil, combinación de los anteriores.
- Group: crea grupos de componentes

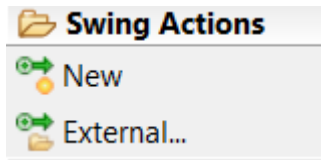


Struts y Springs: son componentes especiales para el BoxLayout



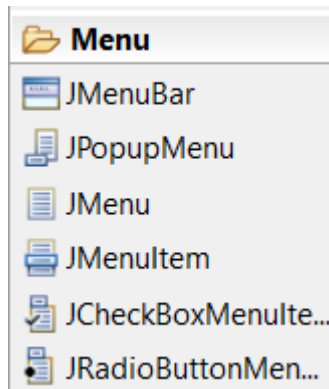
Componentes: los elementos más importantes, los que permitirán al usuario interactuar con la aplicación.

- JLabel: Texto simple informativo, como un título o etiqueta
- JTextField: Caja de texto de una sola línea.
- JComboBox: Lista desplegable de opciones, solo permite seleccionar una opción.
- JButton: Típico botón que permite lanzar un evento.
- JCheckBox: Cuadro para marcar/desmarcar, para seleccionar varios elementos a la vez.
- JRadioButton: Conjunto de elementos en los que se puedes elegir una opción
- JToggleButton: Botón que se queda hundido cuando se le patea.
- JTextArea: Caja de texto que permite escribir varias líneas
- JFormattedTextField: Como el textField pero obliga a seguir un formato
- JPasswordField: Caja de texto que oculta lo que escribes.
- JTextPane: Crear un editor de texto enriquecido
- JEditorPane: Mostrar contenido con formato ya hecho, soporta html.
- JSpinner: Cuadro de texto que permite subir o bajar un dato numérico.
- JList: Muestra una lista fija de elementos, para que el usuario seleccione uno o varios.
- JTable: Organiza la información en filas y columnas, como en excel
- JTree: Muestra datos en una jerarquía como si fuesen carpetas y archivos.
- JProgressBar: barra que se va llenando conforme un proceso se completa.
- JScrollBar: Barra lateral para subir o bajar el contenido de una ventana.
- JSeparator: línea horizontal para dividir visualmente el contenido.
- JSlider: Barra con triángulo que permite ajustar un valor.



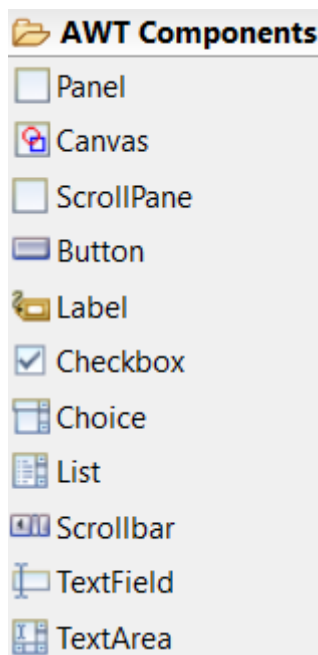
Swing Actions: permite crear lógica que puedas reutilizar

- New: Sirve para crear una nueva orden personalizada, define el nombre de la acción, el ícono y la lógica.
- External: Permite exportar acciones creadas en otras partes del código o en otras librerías.

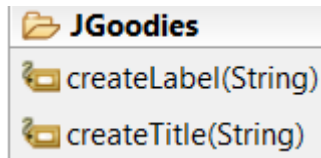


Menu: agrupa componentes que permiten organizar comandos de forma jerárquica.

- JMenuBar: Contenedor principal, franja horizontal que se sitúa en la parte superior de la ventana.
- JPopupMenu: Es un menú que no está pegado arriba, sino que salta en cualquier parte de la pantalla.
- JMenu: pestaña individual que va dentro de la JMenuBar, sirve para desplegar una lista de opciones cuando haces click en él. Puede contener otros JMenu.
- JMenuItem: Botón final dentro de una lista, al pinchar en él se ejecuta la acción.
- JCheckBoxMenuItem: como un JMenuItem, pero con un cuadro de verificación al lado, sirve para activar o desactivar opciones.
- JRadioButtonMenuItem: varias opciones de menú pero sólo puedes elegir una.



AWT Components: AWT es la clase padre, anterior a Swing. A pesar de que están disponibles, no se recomienda su uso y mucho menos mezclarlos con los de Swing.



JGoodies: librería externa muy famosa orientada a que las interfaces se vean más profesionales y limpias.

- `createLabel`: Permite crear una etiqueta optimizada. Configura automáticamente la fuente y el estilo para que se vea mejor.
- `createTitle`: Permite crear títulos de sección. Genera un texto que ya viene con un formato de título, se usa para separar grupos de opciones en una ventana.

¿Cómo abro una ventana desde otra ventana?

Ejemplo: comunicación entre ventanas

¿Cómo podemos hacer que haya comunicación entre ventanas? La idea es sencilla, vamos a crear un botón que nos permita ir desde la ventana actual hasta la ventana que necesitamos; en esa siguiente ventana guardaremos la instancia de la ventana de la que venimos para poder volver. ¿Cómo haremos esto? Guardando como parámetro de la segunda la instancia de la primera ventana.